BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	NGUYỄN LƯƠNG HỒNG BÚP
MSSV / Lớp	3124720009/DNT1241

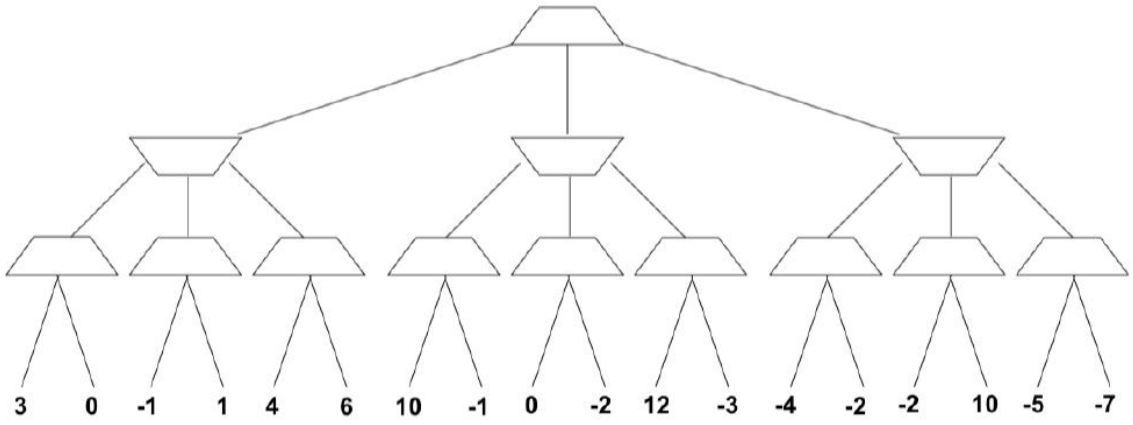
Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Ký hiệu: MIN_1, MIN_2, MIN_3 là ba nút MIN (trái, giữa, phải); mỗi nút MIN có ba nút MAX con: a_1 a_2 a_3 (dưới MIN_1), a_4 a_5 a_6 (dưới MIN_2), a_7 a_8 a_9 (dưới MIN_3).

Tầng MAX (lấy max hai lá): $a_1 = \max(3, 0) = 3$; $a_2 = \max(-1, 1) = 1$; $a_3 = \max(4, 6) = 6$; $a_4 = \max(10, -1) = 10$; $a_5 = \max(0, -2) = 0$; $a_6 = \max(12, -3) = 12$; $a_7 = \max(-4, -2) = -2$; $a_8 = \max(-2, 10) = 10$; $a_9 = \max(-5, -7) = -5$.

Tầng MIN (lấy min ba nút MAX): $MIN_1 = \min(3, 1, 6) = 1$; $MIN_2 = \min(10, 0, 12) = 0$; $MIN_3 = \min(-2, 10, -5) = -5$.

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

Gốc = max(MIN_1, MIN_2, MIN_3) = max(1, 0, −5) = 1. MAX nên chọn nhánh TRÁI (đi vào MIN_1); giá trị trò chơi bằng 1.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Nút	(α, β) khi vào	Diễn biến	Giá trị trả về
Gốc (MAX)	**(−∞, +∞)**	**Sau MIN_1: α = 1; sau MIN_2 và MIN_3 vẫn α = 1**	**1**
MIN_1	**(−∞, +∞)**	**Sau a_1: β = 3; sau a_2: β = 1; a_3 trả 4 (không làm β nhỏ hơn)**	**1**
a_1 (3, 0)	**(−∞, +∞)**	**Lá 3 → α = 3; lá 0 → v = 3**	**3**
a_2 (−1, 1)	**(−∞, 3)**	**Lá −1 → α = −1; lá 1 → α = 1; v = 1 < β = 3 nên chưa cắt**	**1**
a_3 (4, 6)	**(−∞, 1)**	**Lá 4 → α = 4 ≥ β = 1 ⇒ CẮT: lá 6 bị bỏ (X)**	**4**
MIN_2	**(1, +∞)**	**Sau a_4: β = 10; sau a_5: β = 0 ≤ α = 1 ⇒ CẮT a_6 (X)**	**0**
a_4 (10, −1)	**(1, +∞)**	**Lá 10 → α = 10; lá −1 → v = 10**	**10**
a_5 (0, −2)	**(1, 10)**	**Lá 0 → v = 0; lá −2 → v = 0 (0 < β = 10 nên không cắt)**	**0**
a_6 (12, −3)	**—**	**X: không thăm (cắt cả nút, 2 lá 12 và −3)**	**—**
MIN_3	**(1, +∞)**	**Sau a_7: β = −2 ≤ α = 1 ⇒ CẮT a_8, a_9 (X)**	**−2**
a_7 (−4, −2)	**(1, +∞)**	**Lá −4 → v = −4; lá −2 → v = −2**	**−2**
a_8 (−2, 10), a_9 (−5, −7)	**—**	**X: không thăm (cắt 4 lá −2, 10, −5, −7)**	**—**

Kết quả: gốc có α = 1 ⇒ chọn nhánh trái, giá trị 1 (giống Minimax). Các nhánh bị cắt (X): lá thứ hai của a_3; toàn bộ a_6; toàn bộ a_8 và a_9.

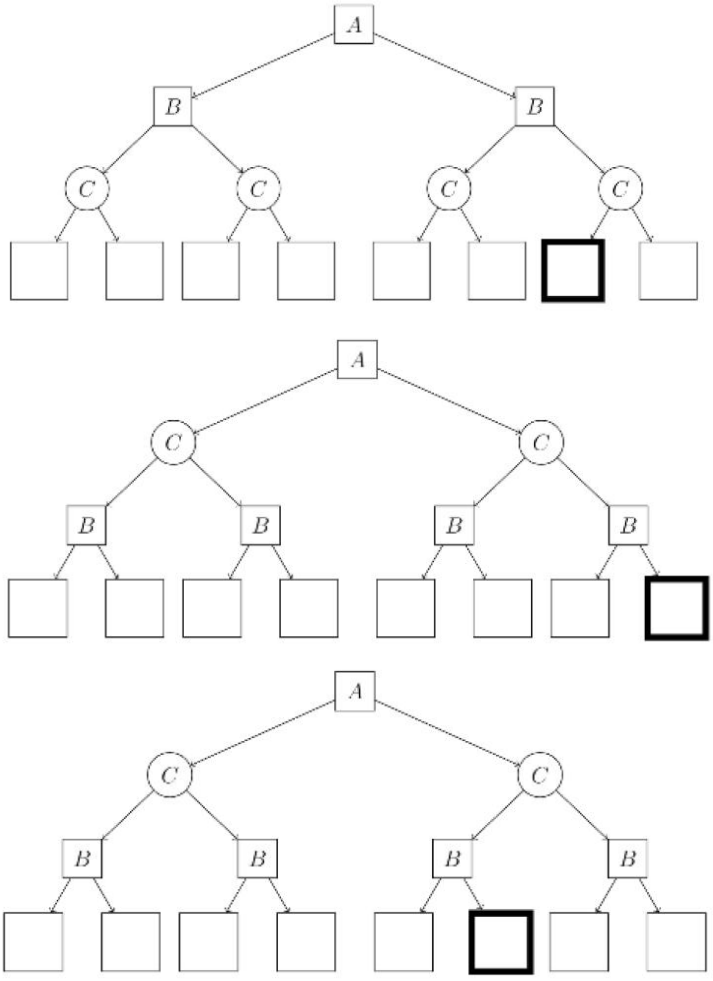
d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

Các lá không cần thăm: 6 (dưới a_3); 12, −3 (dưới a_6); −2, 10 (dưới a_8); −5, −7 (dưới a_9) — tổng cộng 7 lá.

Minimax đánh giá cả 18 lá. Alpha-Beta chỉ đánh giá 11 lá (3, 0, −1, 1, 4, 10, −1, 0, −2, −4, −2), tiết kiệm 7/18 ≈ 39% mà vẫn cho cùng kết quả (giá trị gốc = 1, chọn nhánh trái).

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Đánh số lá từ trái sang phải L1…L8; nút viền đậm là L7 (con đầu của C thứ tư). B trái = max(C_1, C_2) với C_1 = (L1 + L2)/2, C_2 = (L3 + L4)/2; giá trị này trở thành β của A (nút MIN).

Ở B phải (MAX) ta duyệt $C_3 = (L5 + L6)/2$ trước. Nếu $C_3 >$ B trái thì B phải $\geq C_3 > \beta$ nên A (MIN) chắc chắn không chọn B phải ⇒ bỏ qua C_4 ⇒ cắt L7 (và L8).

Điều kiện: $(L5 + L6)/2 > \max\{(L1 + L2)/2, (L3 + L4)/2\}$. Có thể.

Ví dụ: L1…L6 = 2, 4, 0, 2, 6, 8 ⇒ $C_1 = 3$, $C_2 = 1$, B trái = 3; $C_3 = 7 > 3$ ⇒ L7 bị cắt (L7, L8 tùy ý, chẳng hạn 0, 0).

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Nút viền đậm là L8 (lá cuối). Đặt V trái = giá trị C trái = $(B_1 + B_2)/2 = (\max(L1, L2) + \max(L3, L4))/2$ ⇒ β của A = V trái. Ở C phải: $B_3 = \max(L5, L6)$ đã biết, $B_4 = \max(L7, L8) \geq L7$ nên giá trị C phải = $(B_3 + B_4)/2 \geq (B_3 + L7)/2$.

Nếu $(B_3 + L7)/2 >$ V trái thì dù L8 là bao nhiêu, C phải vẫn > V trái ⇒ A (MIN) không chọn ⇒ cắt L8 (tương đương: B_4 có cận $\beta = 2 \cdot$ V trái $- B_3$ và $L7 \geq \beta$).

Điều kiện: $L7 > 2 \cdot$ V trái $- \max(L5, L6)$. Có thể.

Ví dụ: L1…L7 = 3, 0, 3, 0, 10, 0, 10 ⇒ $B_1 = B_2 = 3$, V trái = 3; $B_3 = 10$; $(10 + 10)/2 = 10 > 3$ ⇒ L8 bị cắt (L8 tùy ý).

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Không thể.

Nút viền đậm là L6 (lá thứ hai của B_3, con đầu của C phải). Giá trị C phải = $(B_3 + B_4)/2$ phụ thuộc vào B_4 chưa được duyệt, mà B_4 có thể là bất kỳ số hữu hạn nào (rất âm hoặc rất dương). Khi mới biết L5 thì $B_3 \geq L5$ chỉ là cận dưới của B_3, còn B_4 hoàn toàn không có cận nào, nên không thể kết luận C phải > V trái (hay < V trái) cho mọi L6. Vì không có cận nào để so với β của A, mọi gán giá trị cho các lá khác đều không cho phép bỏ L6 ⇒ không thể cắt tỉa L6.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Trong Minimax, giá trị của nút MAX/MIN chỉ do một con tốt nhất quyết định; chỉ cần biết một con đã vượt ngưỡng α/β thì các con còn lại không thể ảnh hưởng đến kết quả và có thể bỏ qua.

Nút ngẫu nhiên có giá trị là trung bình có trọng số của TẤT CẢ các con, nên mọi con chưa duyệt đều góp vào giá trị. Chỉ cắt được khi chặn được giá trị của các con chưa duyệt (biết khoảng giá trị của lá, hoặc nhờ cận dưới của nút MAX/cận trên của nút MIN đã duyệt một phần như ở trường hợp 1 và 2). Khi giá trị lá không bị chặn (trường hợp 3) thì không thể cắt, nên Expectimax cắt tỉa ít hơn hẳn Minimax.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \le V_E$. ☒ **Đúng** ☐ Sai

Đúng. Với cùng các giá trị con, giá trị nút MIN = min(con) ≤ trung bình có trọng số của các con = giá trị nút ngẫu nhiên. Nút MAX là hàm tăng theo các giá trị con, nên bằng quy nạp từ lá lên gốc ta có $V_M \le V_E$.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☒ **Đúng** ☐ Sai

Đúng. Chính sách Minimax đảm bảo thu được ít nhất V_M trước MỌI cách hành động của đối thủ (kể cả tệ nhất). Mỗi kết quả có thể xảy ra tại nút ngẫu nhiên ứng với một cách "đối thủ" hành động, nên đều cho lợi ích ≥ V_M; do đó lợi ích kỳ vọng (trung bình các kết quả này) cũng ≥ V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☒ **Sai**

Sai. Chính sách Minimax tối ưu cho trường hợp xấu nhất, không tối đa hóa kỳ vọng, nên có thể cho kỳ vọng < V_E. Phản ví dụ: gốc MAX có hai hành động; hành động 1 dẫn tới nút MIN (thành nút ngẫu nhiên đều) với hai lá 10 và 0; hành động 2 dẫn tới nút với hai lá 4 và 4. V_M = max(min(10, 0), min(4, 4)) = 4 nên Minimax chọn hành động 2, kỳ vọng = 4; còn V_E = max((10 + 0)/2, 4) = 5 > 4.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☒ **Minimax với tiện ích trạng thái cuối +1/−1**

☒ **Expectimax với tiện ích trạng thái cuối +1/−1**

☒ **Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua**

☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

Cả ba phương pháp đều có thể dùng, với giả định khác nhau:

• Minimax: coi tính không xác định là "đối thủ" luôn chọn kết quả xấu nhất cho người chơi (mô hình bi quan, không cần biết xác suất); phù hợp khi muốn đảm bảo kết quả tối thiểu hoặc không biết phân phối xác suất.

• Expectimax: cần biết xác suất của các kết quả hành động và cây trò chơi hữu hạn (hoặc giới hạn độ sâu); tối đa hóa tiện ích kỳ vọng = 2·P(thắng) − 1, tức tối đa hóa xác suất thắng.

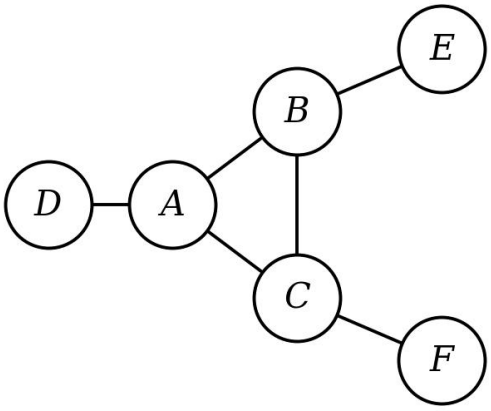
• Value Iteration: mô hình hóa thành MDP với hàm chuyển T(s, a, s′) đã biết, phần thưởng 0 ở trạng thái trung gian và +1/−1 ở trạng thái kết thúc; xử lý được cả trường hợp trạng thái lặp lại/chu trình và đường chơi dài (cần γ < 1 hoặc kết thúc chắc chắn để hội tụ).

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).

Đồ thị có 6 đỉnh A, B, C, D, E, F và 6 cạnh: A–B, A–C, A–D, B–C, B–E, C–F (xem hình). A, B, C tạo thành một tam giác (có chu trình); D, E, F là các đỉnh lá lần lượt treo vào A, B, C.



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Forward Checking chỉ loại giá trị khỏi miền của các láng giềng CHƯA GÁN của A: B, C và D.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

Láng giềng chưa gán của B là C và E (A đã gán nên không xét). Vậy miền của C và E có thể thay đổi.

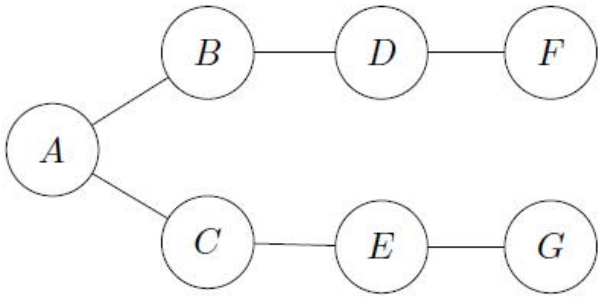
d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

B, C, D, E, F (mọi biến chưa gán). Trước hết B, C, D bị ảnh hưởng trực tiếp vì kề A; sau đó thay đổi lan truyền: miền B đổi kéo theo C, E; miền C đổi kéo theo B, F. Vì đồ thị liên thông nên cả E và F cũng có thể bị đổi gián tiếp.

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

C, E, F. Gán B ảnh hưởng trực tiếp tới láng giềng chưa gán là C và E; thay đổi ở C lan sang F. D chỉ kề A (đã gán, miền A không đổi) nên không thay đổi thêm; A và B đã gán.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

Nhận xét chung: Hình 3 là một cây (thực chất là một đường đi F–D–B–A–C–E–G), nên sau khi chạy Arc Consistency ban đầu, MỌI giá trị còn lại của mọi biến đều mở rộng được thành một nghiệm. Hệ quả: nếu mỗi biến khi được gán chỉ kề với tối đa MỘT biến đã gán trước đó thì luôn có giá trị phù hợp trong miền và không bao giờ phải backtrack.

Giả định: (1) bài toán có nghiệm; (2) "backtrack" = số lần phải hủy một phép gán vì vi phạm ràng buộc hoặc vì Arc Consistency làm rỗng miền; (3) trường hợp xấu nhất: các giá trị tốt luôn bị thử sau cùng, tức mỗi lần thất bại có thể thử tối đa d − 1 giá trị sai trước khi gặp giá trị đúng; (4) giữa các lần chạy Arc Consistency chỉ kiểm tra ràng buộc với các biến đã gán.

f) Thứ tự A → B → C → D → E → F → G

0 lần backtrack. Mỗi biến khi được gán chỉ kề với đúng một biến đã gán (B và C kề A; D kề B; E kề C; F kề D; G kề E) — tức mở rộng từ gốc A ra hai nhánh theo thứ tự cha trước con. Nhờ Arc Consistency ban đầu, luôn có giá trị tương thích với biến cha, nên không bao giờ rơi vào ngõ cụt; các lần chạy Arc Consistency sau 2, 4, 6 biến cũng không làm rỗng miền nào.

g) Thứ tự G → E → C → A → B → D → F

0 lần backtrack. Đây chính là thứ tự đi dọc theo đường G–E–C–A–B–D–F: mỗi biến mới chỉ kề với đúng một biến vừa gán trước nó (E kề G, C kề E, A kề C, B kề A, D kề B, F kề D). Lập luận giống câu f): luôn có giá trị tương thích nên không backtrack.

h) Thứ tự G → F → E → D → C → B → A

Tối đa 3(d − 1) lần backtrack (trường hợp xấu nhất). Thứ tự này gán từ hai đầu của đường vào giữa: biến A (gán cuối) kề với cả B và C đã gán, nên giá trị của hai đầu có thể "không gặp nhau". Arc Consistency chỉ chạy sau 2, 4, 6 biến nên lỗi được phát hiện ở các thời điểm đó:

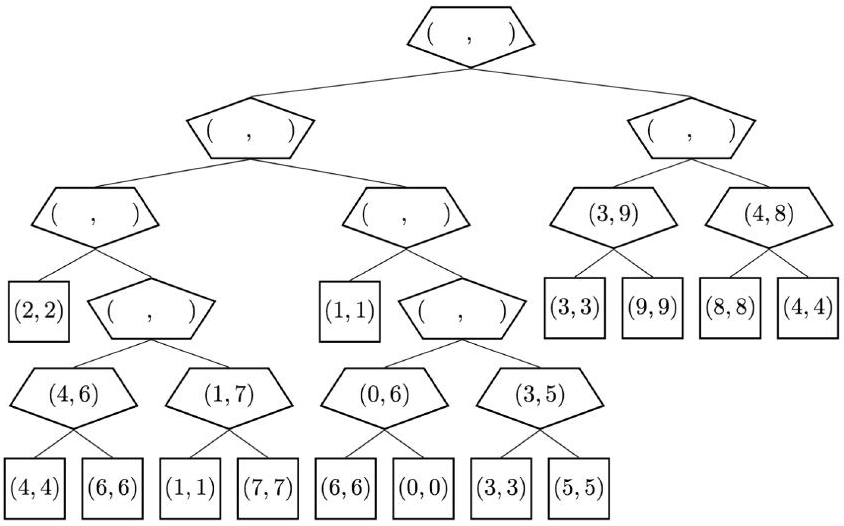
Bước gán	Điều gì có thể sai	Phát hiện khi nào	Backtrack tối đa
G rồi F (2 biến đầu)	G và F không kề nhau nhưng cặp giá trị (g, f) có thể không thể cùng mở rộng thành nghiệm	Arc Consistency sau khi gán F (2 biến)	d − 1 (đổi giá trị F)

E rồi D (biến 3, 4)	**Cặp (e, d) có thể không tương thích qua đường E–C–A–B–D**	**Arc Consistency sau khi gán D (4 biến)**	**d − 1 (đổi giá trị D)**
C rồi B (biến 5, 6)	**Cặp (c, b) có thể không còn giá trị chung cho A (A kề cả B và C)**	**Arc Consistency sau khi gán B (6 biến)**	**d − 1 (đổi giá trị B)**
A (biến 7)	**Miền A đã được thu hẹp theo b và c nên không còn xung đột**	**—**	**0**
Tổng			**3(d − 1)**

Mỗi giá trị đơn lẻ luôn mở rộng được (nhờ Arc Consistency trên cây), nên chỉ cần đổi giá trị của biến vừa gán — không phải lùi sâu hơn; mỗi lần có tối đa d − 1 giá trị sai ⇒ tổng 3(d − 1).

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

Nhận dạng nút: nút hình ngũ giác quay xuống (như (3, 9), (4, 8), (4, 6)…, và cả nút gốc) là nút Casper; nút ngũ giác quay lên (hai con của gốc, và hai nút ở tầng 4) là nút Pacman. Quy tắc: nút Casper: u = min các u con (Pacman tin Casper muốn giảm tiện ích), v = max

các v con (Casper thật sự chọn con tốt nhất cho Pacman). Nút Pacman: u = max các u con; v = v của con có u lớn nhất (Pacman hành động theo niềm tin u).

Nút (mô tả vị trí)	Loại	Tính toán	(u, v)
Nút có hai con (4, 6) và (1, 7)	**Pacman**	**u = max(4, 1) = 4; chọn (4, 6) ⇒ v = 6**	**(4, 6)**
Con phải của nút Casper trái nhất (cạnh lá (2, 2))	**Casper**	**Hai con: lá (2, 2) và (4, 6); u = min(2, 4) = 2; v = max(2, 6) = 6**	**(2, 6)**
Nút có hai con (0, 6) và (3, 5)	**Pacman**	**u = max(0, 3) = 3; chọn (3, 5) ⇒ v = 5**	**(3, 5)**
Nút Casper thứ hai (cạnh lá (1, 1))	**Casper**	**Hai con: lá (1, 1) và (3, 5); u = min(1, 3) = 1; v = max(1, 5) = 5**	**(1, 5)**
Con trái của gốc	**Pacman**	**u = max(2, 1) = 2; chọn (2, 6) ⇒ v = 6**	**(2, 6)**
Con phải của gốc	**Pacman**	**u = max(3, 4) = 4; chọn (4, 8) ⇒ v = 8**	**(4, 8)**
Gốc	**Casper**	**u = min(2, 4) = 2; v = max(6, 8) = 8**	**(2, 8)**

Khác biệt u và v tại nút Casper: u là giá trị theo MÔ HÌNH của Pacman (Pacman cho rằng Casper đối kháng nên lấy min), còn v là giá trị theo HÀNH VI THỰC của Casper (Casper thân thiện nên lấy max). Vì vậy Pacman tin ván cờ có giá trị 2, trong khi thực tế (Casper chọn nhánh phải) tiện ích thu được là 8.

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Để tính v, Casper phải biết Pacman chọn con nào (so sánh u), rồi chỉ cần v của con được chọn. Có thể bỏ (đánh dấu X):

• Lá (7, 7): ở nút Pacman có hai con (4, 6) và (1, 7), sau khi (4, 6) cho u = 4, nút (1, 7) có lá đầu (1, 1) ⇒ u ≤ 1 < 4 nên Pacman chắc chắn chọn (4, 6); lá (7, 7) không ảnh hưởng.

• Toàn bộ nhánh con phải của nút Casper thứ hai (nút (1, 5)), gồm hai nút (0, 6) và (3, 5) cùng bốn lá (6, 6), (0, 0), (3, 3), (5, 5): ở nút Pacman con trái của gốc, nhánh đầu có u = 2; nhánh thứ hai sau lá (1, 1) đã có u ≤ 1 < 2 nên Pacman chọn nhánh đầu, phần còn lại không cần duyệt.

Các nhánh còn lại đều cần duyệt (nhánh phải của gốc phải duyệt cả hai nút (3, 9) và (4, 8) để so sánh u = 3 với u = 4; nếu cho phép đảo thứ tự duyệt thì lá (9, 9) cũng có thể bỏ, nhưng theo đúng thứ tự trái sang phải thì không).

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Khi đó Pacman cũng coi Casper là người tối đa hóa nên mọi nút đều lấy max: u = v tại mọi nút và giá trị cây bằng lá lớn nhất, tức 9 (đường gốc → nhánh phải → (3, 9) → lá (9, 9)). Giá trị tăng từ 8 (kết quả thực khi Pacman hiểu nhầm; niềm tin của Pacman chỉ là 2) lên 9, vì Pacman sẽ chọn (3, 9) thay vì (4, 8) ở nhánh phải.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	**Hệ số nhánh b hữu hạn (có nghiệm ở độ sâu hữu hạn); đúng cho cả tìm kiếm trên cây và trên đồ thị.**	**Chi phí mọi bước bằng nhau (hoặc chi phí không giảm theo độ sâu) — khi đó nghiệm nông nhất cũng là rẻ nhất.**
Depth-First Search (DFS)	**Chỉ khi không gian tìm kiếm hữu hạn (độ sâu hữu hạn) và dùng tìm kiếm trên đồ thị (có tập đã thăm) để tránh chu trình. Tìm kiếm trên cây với đồ thị có chu trình hoặc độ sâu vô hạn thì không đầy đủ.**	**Không tối ưu (kể cả khi chi phí đều): có thể gặp nghiệm sâu hơn/đắt hơn trước.**
Iterative Deepening Search (IDS)	**Hệ số nhánh b hữu hạn (có nghiệm ở độ sâu hữu hạn); đầy đủ cả khi không gian vô hạn.**	**Chi phí mọi bước bằng nhau (tìm nghiệm nông nhất); không tối ưu khi chi phí bước khác nhau.**
A* Search	**b hữu hạn và chi phí mỗi bước ≥ ε > 0 (và có nghiệm với chi phí hữu hạn).**	**Tìm kiếm trên cây: heuristic chấp nhận được (admissible, h ≤ h*). Tìm kiếm trên đồ thị: heuristic nhất quán (consistent), hoặc admissible nhưng phải cho phép mở lại nút khi tìm được đường rẻ hơn.**

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

Chi phí bước: BFS và IDS chỉ cực tiểu hóa số bước (độ sâu) nên chỉ tối ưu khi mọi bước có cùng chi phí; nếu chi phí khác nhau, nghiệm nông nhất có thể đắt hơn. Chi phí bằng 0 hoặc tiến về 0 tạo ra chu trình/dãy vô hạn các nút có f không tăng, làm A* mất tính đầy đủ (cần ≥ ε > 0); chi phí âm làm hỏng cả A* lẫn tính tối ưu.

Heuristic: nếu h đánh giá quá cao (không admissible), A* có thể bỏ qua đường tối ưu và trả về nghiệm kém. Trên tìm kiếm đồ thị, h chỉ admissible mà không nhất quán có thể làm A* đóng một nút khi chưa đạt g tối ưu (nút không được mở lại) nên trả về nghiệm không tối ưu. Hệ số nhánh vô hạn làm mất tính đầy đủ của mọi thuật toán trên.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

Ký hiệu: C* là chi phí tối ưu cũ; g(·) là chi phí đường đi tốt nhất đã biết từ trạng thái đầu.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Chi phí tối ưu CÓ THỂ thay đổi (tăng tối đa n) và đường đi có thể đổi. Đường cũ nay tốn C* + n; các đường không qua X → Y giữ nguyên chi phí (≥ C*), nên chi phí tối ưu mới = min(C* + n, chi phí đường tốt nhất khác) ∈ [C*, C* + n].

Cập nhật: không cần chạy lại từ đầu. Các nút có g không dựa vào cạnh X → Y vẫn đúng; chỉ cần cộng thêm n vào g của Y và các nút con cháu của Y trong cây tìm kiếm (hoặc đưa chúng trở lại frontier), rồi tiếp tục A* từ frontier hiện có (heuristic vẫn nhất quán vì chi phí chỉ tăng) cho đến khi nút đích được lấy ra.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Chi phí tối ưu giảm đúng n, đường đi KHÔNG đổi: C*mới = C* − n. Đường cũ giảm n; mọi đường khác giảm tối đa n (một đường đơn qua cạnh tối đa một lần) và vốn ≥ C* nên vẫn ≥ C* − n. Chỉ cần cập nhật chi phí, không cần tìm lại.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Không đổi: đường tối ưu cũ không dùng cạnh này nên vẫn có chi phí C*, còn các đường khác chỉ đắt hơn hoặc bằng. Không cần làm gì.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

CÓ THỂ thay đổi (giảm, nhưng không quá n): chi phí tối ưu mới ∈ [C* − n, C*]. Đường mới qua X → Y có thể rẻ hơn C*.

Cập nhật: tính g′(Y) = g(X) + (c − n). Nếu g′(Y) ≥ g(Y) hiện có thì không có gì thay đổi. Nếu nhỏ hơn thì mở lại Y với g mới, lan truyền cải thiện cho các nút con cháu đã mở, rồi tiếp tục A* đến khi f nhỏ nhất ở frontier ≥ chi phí đích tốt nhất (giả sử heuristic vẫn chấp nhận được).

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Không đổi: cạnh nằm ngoài đường tối ưu và chỉ đắt hơn, nên đường tối ưu và chi phí C* giữ nguyên; A* không cần xử lý thêm.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

CÓ THỂ thay đổi (giảm, ≥ C* − n). Vì cạnh chưa được khám phá nên trước đây nút chứa nó có f ≥ C*; nay đường qua cạnh có f mới = g(X) + (c − n) + h(Y) có thể < C*.

Cập nhật: nếu f mới ≥ C* thì bỏ qua (không đổi). Nếu f mới < C* thì đưa Y vào frontier với g mới và tiếp tục A* — chỉ cần mở các nút có f nhỏ hơn chi phí đích tốt nhất hiện có, không phải chạy lại từ đầu (giả sử heuristic vẫn chấp nhận được).

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

- A: act(percept) → f_A()
- B: act(percept) → f_B(percept)
- C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

B và C (không phải A). Quan sát đầy đủ nên percept chính là trạng thái; môi trường xác định, tĩnh, một tác tử nên hành động tối ưu chỉ phụ thuộc trạng thái hiện tại: f_B(percept) = π*(trạng thái) là đủ; C bao hàm B (chỉ cần bỏ qua lịch sử). A không được vì f_A() không nhìn percept, trong khi hành động tối ưu thay đổi theo trạng thái và theo môi trường trong lớp.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Chỉ C. Percept hiện tại không đủ để biết trạng thái thật; vì môi trường xác định và tĩnh, lịch sử percept (cùng các hành động tất định mà chính f_C đã chọn) cho phép suy ra tập trạng thái có thể đúng, nên f_C(lịch sử percept) có thể hành động tối ưu. B chỉ nhìn percept hiện tại nên không phân biệt được hai tình huống cùng percept nhưng khác lịch sử (ví dụ cần nhớ một tín hiệu đã thấy trước đó); A càng không.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Chỉ C. Cần nhớ lịch sử percept để duy trì niềm tin (belief state) về trạng thái; hành động tối đa hóa tiện ích kỳ vọng là hàm của niềm tin này, và với một tác tử, môi trường tĩnh thì tồn tại một hàm f_C tất định cho hành động tối ưu (nên các hành động trong quá khứ vẫn tính lại được từ lịch sử percept). Tính ngẫu nhiên của môi trường không làm cho B hay A đủ, vì chúng không có thông tin lịch sử.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng. Ví dụ: môi trường mà mọi hành động đều cho cùng một tiện ích (ví dụ chỉ có một hành động hợp lệ, hoặc phần thưởng luôn bằng 0 bất kể hành động). Khi đó mọi tác tử, kể cả tác tử chọn hành động ngẫu nhiên hay loại A, B, C bất kỳ, đều đạt tiện ích tối đa nên đều hành động hợp lý.

— Hết —